

Lab 12: Software-Defined Radio (SDR) Data Collection and Preprocessing

Project Title: *RF Modulation Classification Using Software Defined Radio (SDR)*

Group Members Name: Syed Ali Nisar Shah (ss007136) & Syed Muhammad Asad Ali (sa09475)

Section 1: Data Collection Methodology

The objective of this project is to design and implement a system capable of identifying and classifying radio frequency (RF) signals based on their underlying modulation schemes using a Software Defined Radio (SDR) and machine learning techniques.

In modern wireless environments, countless devices transmit using a variety of modulation types such as AM, FM, QPSK, OOK, etc. Correctly recognizing these modulation schemes is essential for applications in spectrum monitoring, cognitive radio, adaptive communication systems, and interference analysis and used as the foundation for building more advanced systems.

1.1 Hardware Configuration

- **Signal Type:** We decided on three signals Air Traffic Control (ATC), Broadcast FM and Car Keyfob Signals. Each of these signals represent a different type of modulation scheme. ATC represents AM, FM represents FM and Car Keyfob represents OOK Modulation.
- **HackRF Settings:** The exact parameters used on the HackRF One:
 - **Center Frequency (f_c):** The precise frequency tuned to signals were the following:
 - FM Signals: 107.0 & 106.2 MHz;
 - ATC: 121.806 MHz;
 - Car Keyfob: 314.0 MHz
 - **The sample rate (f_s)** at which the I/Q samples were captured was set at 20MS/s for all signals.
 - **Gain Settings:** The gain we set was 2.0 dB by trial and error for all signals as it balanced data capture and noise attenuation.
- A good antenna is important since if the antenna is not tuned to the frequency, HackRF One will receive less energy and the signal will appear weak or missing. The Antenna provided was ANT500. It has a wide frequency range of 75Mhz – 1Ghz even though HackRF One itself operates from 1Mhz to 6Mhz. Many signals fall in this range so it works well for most basic SDR tasks.
 - 1- The antenna was placed on the university rooftop to obtain clearer ATC signals taking advantage of strong RF activity near nearby airport.
 - 2- FM signals are strong and can be caught from anywhere around the university and the house.

3- Car keyfob signals were stronger if the device was present in the same room so antenna was placed in the same room for strong RF signals.

1.2 Software Stack

- **Capture Tool:** GQRX 2.17.7 for raw data capture
- **Commands/Flowgraph:** We used GQRX's IQ recording and replay tool for capturing raw signals/data. We also used GNU Radio Companion for custom playback flow graph for FM signals.

1.3 Dataset Description

- **Volume:** Our dataset is about 106 GB in size. It includes the raw data that we captured across different modulation schemes using HackRF One, the different chunks we made from the original dataset, various spectrograms and numpy arrays we extracted, and any copies needed for testing.
- **Classes/Labels:** Different classes(labels) we decided in the dataset are: Amplitude Modulation (AM), Frequency Modulation (FM) and On-Off Keying (OOK).

Section 2: Data Preprocessing and Feature Extraction

This section describes the steps taken to transform the raw I/Q samples into a format usable by an ML/DL model.

2.1 Initial Data Handling

- **Format Conversion:**

The raw radio frequency data was stored in binary files (.raw) consisting of interleaved In-phase (I) and Quadrature (Q) samples. These files were ingested into the Python environment using the NumPy library. Specifically, the data was read as 32-bit complex floating-point numbers (complex64), where the real component represents the I-channel and the imaginary component represents the Q-channel.

- **Normalization:**

Before the CNN training phase, a logarithmic normalization was applied to the frequency domain magnitude to convert the scale to Decibels (dB) for cleaner visualization.

We did normalize the data between [0,1] but it was not done until the training phase of the CNN.

2.2 Segmentation/Windowing

- **Window Size:**

To prepare the continuous signal for time-frequency analysis, the data was segmented using a windowing approach. The window size was set to **2,048 samples** per segment (nperseg=2048). This segment size was chosen to provide a balance between frequency resolution and time resolution given the 20 MHz sample rate.

2.3 Feature Extraction

We used the Deep Learning approach by going with Convolution Neural Network (CNN).

The raw I/Q data was transformed into .npy spectrograms to be used as inputs for the Convolutional Neural Network (CNN). This was achieved using the Short-Time Fourier Transform (STFT).

The transformation process involved the following steps:

1. **STFT Computation:** The complex I/Q data was processed using the `scipy.signal.spectrogram` function.
2. **Magnitude Extraction:** The complex output of the FFT was converted to magnitude:
$$\sqrt{(\text{Real})^2 + (\text{Imag})^2}$$
3. **Logarithmic Scaling:** The linear magnitude spectrogram (S_{xx}) was converted to a logarithmic dB scale S_{dB} using the formula:
$$S_{dB} = 20 \log_{10}(S_{xx} + \epsilon)$$

Note: An epsilon $\epsilon = 1 \times 10^{-12}$ was added to prevent mathematical errors when taking the log of zero.
4. **Result:** The resulting matrix was saved as .npy file and grayscale image for visualization to provide raw pixel data for the CNN.

The following parameters were utilized for the STFT:

- **Window Type:** Hann window (to minimize spectral leakage).
- **FFT Size (NFFT):** 2,048 bins.
- **Overlap:** 1,024 samples (50% overlap between consecutive frames).
- **Scaling:** Density.

Section 3: Data Organization and Validation

3.1 Data Structure:

The .npy files in the final dataset have the following shape and datatype:

- **Shape:** (224, 224) (224 x 224 arrays)
- **Data Type:** Little-endian 32-bit floating-point

3.2 Data Split:

This is the dataset split for our project:

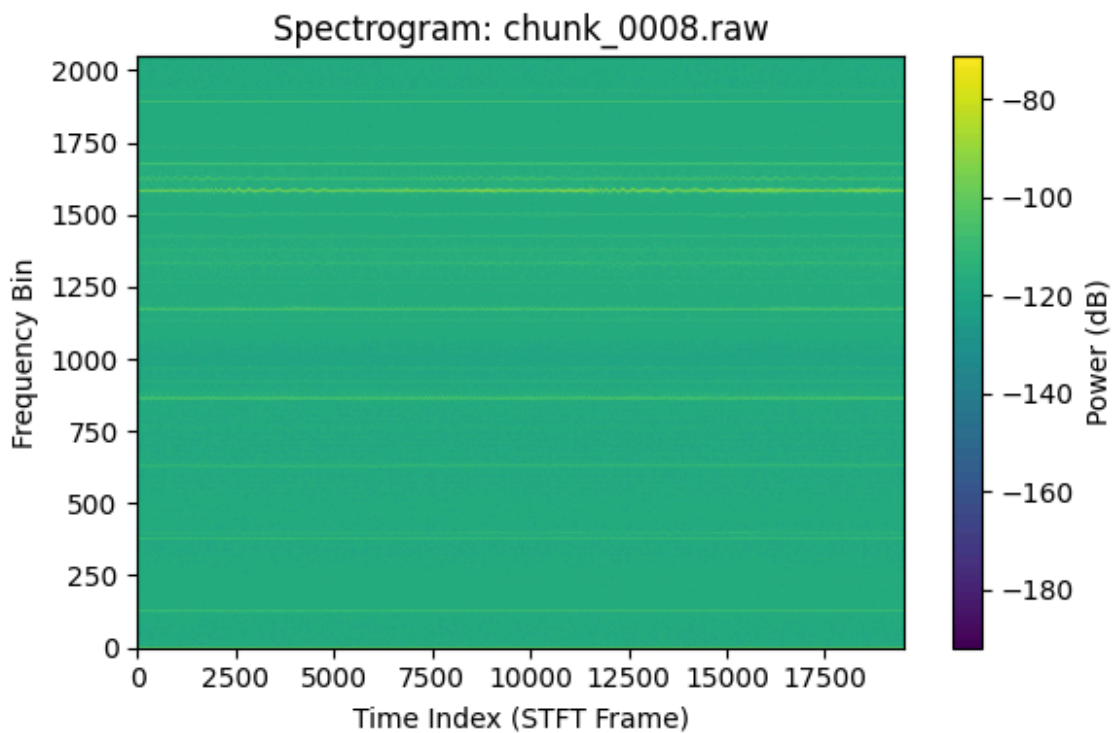
- **Training set ratio** = 70% (615 samples across classes)
- **Validation set ratio** = 15% (130 samples across classes)
- **Test set ratio** = 15% (132 samples across classes)

3.3 Visualization (Required):

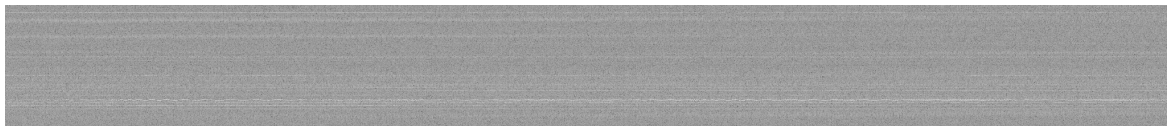
We have spectrograms from two stages: from the 1-second chunks we made to standardize the dataset and then the 224x224 images with significant power we extracted from these chunks to train the CNN.

I. Spectrograms of 1-second chunks of RAW data:

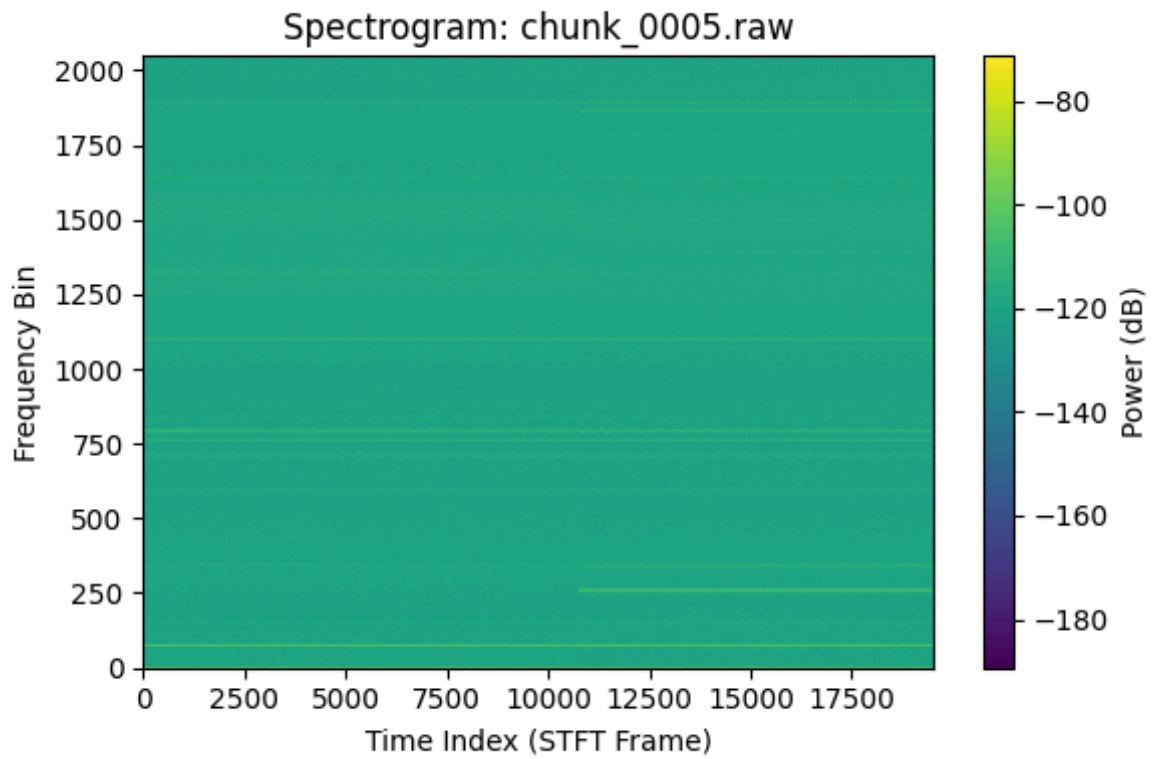
1. FM (chunk_0008 from our FM dataset):
 - a. Color map:



- a. Grayscale spectrogram:



2. AM (chunk_0005 from AM dataset):
 - a. Color map:

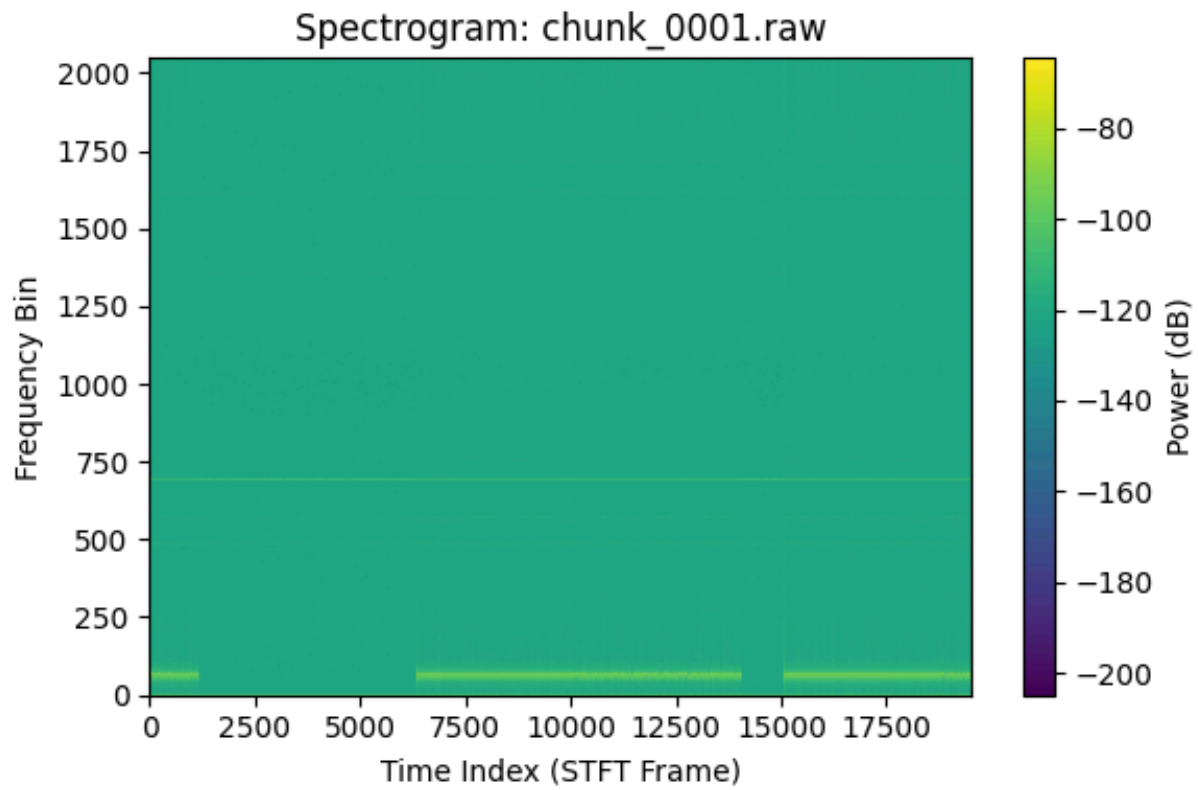


b. Grayscale spectrogram:

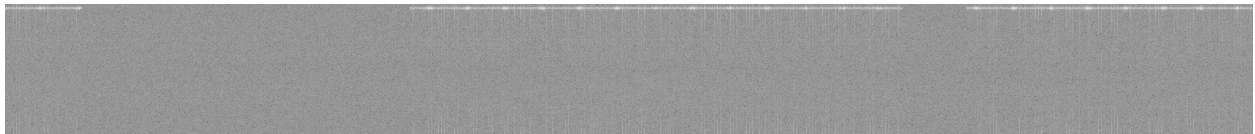


3. OOK (chunk_0001 of RAW OOK dataset):

a. Color map:

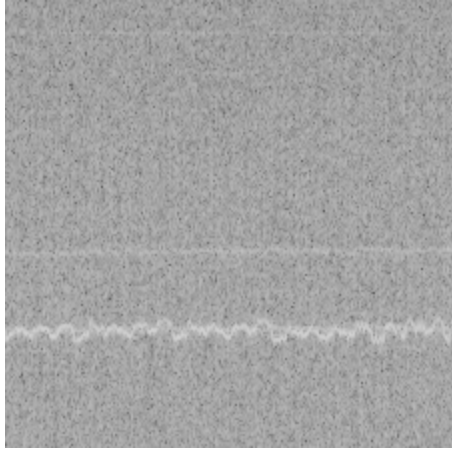


b. Grayscale spectrogram:

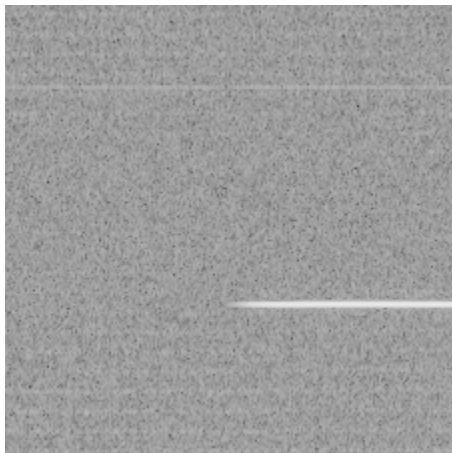


II. Extracted 224x224 high-power signals for CNN training, validation, and testing that contain the actual modulated signals:

1. FM:



2. AM:



3. OOK:

